

Experiment 1 :

Problem statement :

Construct an AVL Tree for a given set of elements. Implement Insertion and Deletion operations with automatic balancing, and display the tree elements using In-order Traversal to verify the sorted order.

Aim :

To construct an **AVL Tree** for a given set of elements, implement **Insert** and **Delete** operations, and display the elements of the tree using **In-order Traversal**

Algorithm :

Node structure:

- Each node has: data, left, right, height
 - $\text{height} = 1 + \max(\text{height}(\text{left}), \text{height}(\text{right}))$
 - NULL node height = 0
-

Helper Functions

- **getHeight(node):** return height or 0 if NULL
 - **getBalance(node):** return $\text{height}(\text{left}) - \text{height}(\text{right})$
 - **RightRotate(y):** Rotate right, update heights, return new root
 - **LeftRotate(x):** Rotate left, update heights, return new root
 - **minValueNode(node):** Go left till NULL, return that node
-

Insert(key):

1. Normal BST insertion
 2. Update height
 3. Find balance factor
 4. Rebalance if needed:
 - LL: RightRotate
 - LR: LeftRotate left child → RightRotate
 - RR: LeftRotate
 - RL: RightRotate right child → LeftRotate
-

Delete(key):

1. Normal BST delete
 - No child → delete node
 - One child → return child
 - Two children → copy inorder successor, delete it
 2. Update height
 3. Find balance factor
 4. Rebalance like insert
-

In-order Traversal:

- Traverse Left → Print root → Traverse Right (gives sorted order)

Code :

```
#include <stdio.h>
#include <stdlib.h>
typedef struct node{
    int data,height;
    struct node *left,*right;
}Node;

int height(Node *N){
    if (N == NULL)
        return 0;
    return N->height;
}

Node *newNode(int data){
    Node *node = (Node *) malloc(sizeof(Node));
    node->data = data;
    node->left = NULL;
    node->right = NULL;
    node->height = 1;
    return (node);
}

Node *rightRotate(Node *y){
    Node *x = y->left;
    Node *T2 = x->right;
```

```

x->right = y;
y->left = T2;
y->height = max(height(y->left), height(y->right)) + 1;
x->height = max(height(x->left), height(x->right)) + 1;
return x;
}

```

```

Node *leftRotate(Node *x){
    Node *y = x->right;
    Node *T2 = y->left;
    y->left = x;
    x->right = T2;
    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;
    return y;
}

```

```

int getBalance(Node *N){
    if (N == NULL)
        return 0;
    return height(N->left) - height(N->right);
}

```

```

Node *insertNode(Node *node, int data){
    int balance;

    if (node == NULL)
        return (newNode(data));
    if (data < node->data)
        node->left = insertNode(node->left, data);
    else if (data > node->data)
        node->right = insertNode(node->right, data);
    else
        return node;

    node->height = 1 + max(height(node->left), height(node->right));
    balance = getBalance(node);

    if (balance > 1 && data < node->left->data)
        return rightRotate(node);
    if (balance < -1 && data > node->right->data)
        return leftRotate(node);
    if (balance > 1 && data > node->left->data) {
        node->left = leftRotate(node->left);
    }
}

```

```

        return rightRotate(node);
    }
    if (balance < -1 && data < node->right->data) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }
    return node;
}

```

```
Node *minValueNode(Node *node){
    Node *current = node;
    while (current->left != NULL)
        current = current->left;
    return current;
}
```

```
Node *deleteNode(Node *root, int data){
    int balance;

    if (root == NULL)
        return root;
    if (data < root->data)
        root->left = deleteNode(root->left, data);
    else if (data > root->data)
        root->right = deleteNode(root->right, data);
    else {
        if ((root->left == NULL) || (root->right == NULL)) {
            Node *temp = root->left ? root->left : root->right;
            if (temp == NULL) {
                temp = root;
                root = NULL;
            } else
                *root = *temp;
            free(temp);
        } else {
            Node *temp = minValueNode(root->right);
            root->data = temp->data;
            root->right = deleteNode(root->right, temp->data);
        }
    }
    if (root == NULL)
        return root;
    root->height = 1 + max(height(root->left),
                           height(root->right));
```

```

balance = getBalance(root);
if (balance > 1 && getBalance(root->left) >= 0)
    return rightRotate(root);
if (balance > 1 && getBalance(root->left) < 0) {
    root->left = leftRotate(root->left);
    return rightRotate(root);
}
if (balance < -1 && getBalance(root->right) <= 0)
    return leftRotate(root);
if (balance < -1 && getBalance(root->right) > 0) {
    root->right = rightRotate(root->right);
    return leftRotate(root);
}
return root;
}

```

// Print the tree

```

void printTree(Node *root){
    if (root != NULL) {
        printTree(root->left);
        printf("%d ", root->data);
        printTree(root->right);
    }
}

```

```

void main(){
    Node *root = NULL;
    int a[]={22,14,72,44,25,63,98},i;
    clrscr();
    for(i=0;i<sizeof(a)/2;i++)
        root = insertNode(root, a[i]);
    printf("AVL Tree: ");
    printTree(root);
    root = deleteNode(root, 25);
    printf("\nAfter deletion of 25: ");
    printTree(root);
    getch();
}

```

Output :

AVL Tree: 14 22 25 44 63 72 98

After deletion of 25 : 14 22 44 63 72 98

Experiment 2:

Problem statement :

To construct a **B-Tree of order 5** using a given set of 10 random elements stored in an array, and to implement **Searching**, **Insertion**, and **Deletion** operations on the B-Tree.

Aim :

States the **objective** briefly. Example: *"To construct a B-Tree of order 5 and implement Searching, Insertion, and Deletion operations."*

Key Points (Order 5)

- Max keys = 4, Max children = 5
 - Min keys = 2 (except root, min = 1)
-

1. Construct B-Tree

- Start with empty tree
 - Insert each element using **BTree_Insert**
-

2. Search(key)

- Traverse keys in node
 - If key found → return node
 - If leaf → return NULL
 - Else → search in child[i]
-

3. Insert(key)

- If root full → create new root, split root, insert key
- Else → InsertNonFull(root, key)

InsertNonFull(node, key):

- If leaf → insert key at correct position
- Else → go to child, split if full, recurse

SplitChild(parent, i):

- Split full child into two nodes
- Move middle key up to parent

4. Delete(key)

- If key in node:
 - Leaf → delete directly
 - Internal → replace by predecessor/successor or merge children
 - If key not in node:
 - If child has 1 key → borrow/merge
 - Recurse into correct child
-

5. Display (In-order)

- Recursively traverse: Left children → Key → Right children

CODE :

// Online C compiler to run C program online

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define ORDER 5
```

```
#define MAX ORDER-1
```

```
#define MIN ORDER/2-1
```

```
typedef struct Node {  
    int key[MAX], count, leaf;  
    struct Node *child[ORDER];  
}Node;
```

```
Node *root = NULL;
```

```
Node* create(int leaf) {  
    int i;  
    Node *n = (Node*)malloc(sizeof(Node));  
    n->count = 0; n->leaf = leaf;  
    for (i = 0; i < ORDER; i++)  
        n->child[i] = NULL;  
    return n;  
}
```

```

void traverse(Node *t) {
    int i;
    for (i = 0; i < t->count; i++) {
        if (!t->leaf) traverse(t->child[i]);
        printf("%d ", t->key[i]);
    }
    if (!t->leaf) traverse(t->child[i]);
}

```

```

Node* search(Node *t, int k) {
    int i = 0;
    while (i < t->count && k > t->key[i]) i++;
    if (i < t->count && t->key[i] == k) return t;
    if (t->leaf) return NULL;
    return search(t->child[i], k);
}

```

```

void split(Node *p, int i, Node *y) {
    int j;
    struct Node *z = create(y->leaf);

    z->count = MIN;
    for (j = 0; j < MIN; j++)
        z->key[j] = y->key[j + MIN + 1];

    if (!y->leaf) {
        for (j = 0; j <= MIN; j++)
            z->child[j] = y->child[j + MIN + 1];
    }

    y->count = MIN;

    for (j = p->count; j >= i + 1; j--)
        p->child[j + 1] = p->child[j];
    p->child[i + 1] = z;

    for (j = p->count - 1; j >= i; j--)
        p->key[j + 1] = p->key[j];

    p->key[i] = y->key[MIN];
    p->count++;
}

```

```

void insertNonFull(Node *x, int k) {
    int i = x->count - 1;
    if (x->leaf) {
        while (i >= 0 && x->key[i] > k) {
            x->key[i+1] = x->key[i];
            i--;
        }
        x->key[i+1] = k;
        x->count++;
    } else {
        while (i >= 0 && x->key[i] > k) i--;
        i++;
        if (x->child[i]->count == MAX) {
            split(x, i, x->child[i]);
            if (x->key[i] < k) i++;
        }
        insertNonFull(x->child[i], k);
    }
}

```

```

void insert(int k) {
    int i;
    if (!root) {
        root = create(1);
        root->key[0] = k; root->count = 1;
    } else {
        if (root->count == MAX) {
            Node *s = create(0);
            s->child[0] = root;
            split(s, 0, root);
            i = 0;
            if (s->key[0] < k) i++;
            insertNonFull(s->child[i], k);
            root = s;
        } else insertNonFull(root, k);
    }
}

```

/* ----- Deletion Part (Short Form) ----- */

```

int findKey(Node *x, int k) {
    int idx = 0;
    while (idx < x->count && x->key[idx] < k) ++idx;
    return idx;
}

```

```

int getPred(Node *x, int idx) {
    struct Node *cur = x->child[idx];
    while (!cur->leaf) cur = cur->child[cur->count];
    return cur->key[cur->count - 1];
}

int getSucc(Node *x, int idx) {
    struct Node *cur = x->child[idx+1];
    while (!cur->leaf) cur = cur->child[0];
    return cur->key[0];
}

void merge(Node *x, int idx) {
    struct Node *c = x->child[idx], *s = x->child[idx+1];
    int i;
    c->key[MIN] = x->key[idx];
    for (i = 0; i < s->count; i++) c->key[i+MIN+1] = s->key[i];
    if (!c->leaf)
        for (i = 0; i <= s->count; i++) c->child[i+MIN+1] = s->child[i];
    for (i = idx+1; i < x->count; i++) x->key[i-1] = x->key[i];
    for (i = idx+2; i <= x->count; i++) x->child[i-1] = x->child[i];
    c->count += s->count + 1;
    x->count--;
    free(s);
}

void fill(Node *x, int idx) {
    int i;
    if (idx != 0 && x->child[idx-1]->count >= MIN+1) {
        struct Node *c = x->child[idx], *s = x->child[idx-1];
        for (i = c->count-1; i >= 0; i--) c->key[i+1] = c->key[i];
        if (!c->leaf)
            for (i = c->count; i >= 0; i--) c->child[i+1] = c->child[i];
        c->key[0] = x->key[idx-1];
        if (!c->leaf) c->child[0] = s->child[s->count];
        x->key[idx-1] = s->key[s->count-1];
        c->count++; s->count--;
    } else if (idx != x->count && x->child[idx+1]->count >= MIN+1) {
        struct Node *c = x->child[idx], *s = x->child[idx+1];
        c->key[c->count] = x->key[idx];
        if (!c->leaf) c->child[c->count+1] = s->child[0];
        x->key[idx] = s->key[0];
        for (i = 1; i < s->count; i++) s->key[i-1] = s->key[i];
    }
}

```

```

        if (!s->leaf)
            for (i = 1; i <= s->count; i++) s->child[i-1] = s->child[i];
            c->count++; s->count--;
    } else {
        if (idx != x->count) merge(x, idx);
        else merge(x, idx-1);
    }
}

```

```

void deleteKey(Node *x, int k) {
    int idx = findKey(x, k), i, pred, flag;
    if (idx < x->count && x->key[idx] == k) {
        if (x->leaf) {
            for (i = idx+1; i < x->count; i++) x->key[i-1] = x->key[i];
            x->count--;
        } else {
            if (x->child[idx]->count >= MIN+1) {
                pred = getPred(x, idx);
                x->key[idx] = pred;
                deleteKey(x->child[idx], pred);
            } else if (x->child[idx+1]->count >= MIN+1) {
                int succ = getSucc(x, idx);
                x->key[idx] = succ;
                deleteKey(x->child[idx+1], succ);
            } else {
                merge(x, idx);
                deleteKey(x->child[idx], k);
            }
        }
    } else {
        if (x->leaf) return;
        flag = (idx == x->count);
        if (x->child[idx]->count < MIN+1) fill(x, idx);
        if (flag && idx > x->count) deleteKey(x->child[idx-1], k);
        else deleteKey(x->child[idx], k);
    }
}

```

/* ----- Main ----- */

```

void main() {
    int arr[] = {15, 3, 9, 20, 6, 1, 25, 30, 12, 18};
    int i, k;
    for (i = 0; i < 10; i++)
        insert(arr[i]);
}

```

```

clrscr();
printf("B-Tree after insertion:\n");
traverse(root); printf("\n");

printf("Enter key to search: ");
scanf("%d", &k);
if (search(root, k))
    printf("Key %d found.\n", k);
else
    printf("Key %d not found.\n", k);

printf("Enter key to delete: ");
scanf("%d", &k);
deleteKey(root, k);
printf("B-Tree after deletion:\n");
traverse(root); printf("\n");
getch();
}

```

Output :

B-Tree after insertion:

1 3 6 9 12 15 18 25

Enter key to search: 9

Key 9 found.

Enter key to delete: 3

B-Tree after deletion:

1 6 9 12 15 18 25

Experiment 3 :

Problem Statement:

"Construct Min and Max Heap using arrays, delete any element and display the content of the Heap."

Aim

To implement Min Heap and Max Heap using arrays. The program should allow inserting elements, deleting any element, and displaying the heap content after each operation.

Algorithm

Max Heap Construction

1. Start with an array of elements.
2. Insert each element into the heap.
3. For each insertion, "heapify up" to maintain the Max Heap property:
 - Parent node must be greater than its children.

Min Heap Construction

1. Start with an array of elements.
2. Insert each element into the heap.
3. For each insertion, "heapify up" to maintain the Min Heap property:
 - Parent node must be smaller than its children.

Deletion of an Element (from Min/Max Heap)

1. Find the index of the element to delete.
2. Replace it with the last element in the heap.
3. Reduce heap size by 1.
4. Apply heapify (down) to restore heap property.

Display

- Traverse array from index 0 to size-1 and print heap contents.

Sample code:

```
#include <stdio.h>
#define MAX 50
int size=0;
int i;

// Swap function
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify for Max Heap
void heapifyMax(int arr[], int n, int i) {
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapifyMax(arr, n, largest);
    }
}

// Heapify for Min Heap
void heapifyMin(int arr[], int n, int i) {
    int smallest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if (left < n && arr[left] < arr[smallest])
        smallest = left;

    if (right < n && arr[right] < arr[smallest])
        smallest = right;
```

```

        if (smallest != i) {
            swap(&arr[i], &arr[smallest]);
            heapifyMin(arr, n, smallest);
        }
    }

// Build Max Heap
void buildMaxHeap(int arr[], int n) {
    for (i = n/2 - 1; i >= 0; i--)
        heapifyMax(arr, n, i);
}

// Build Min Heap
void buildMinHeap(int arr[], int n) {
    for (i = n/2 - 1; i >= 0; i--)
        heapifyMin(arr, n, i);
}

// Delete element from Max Heap
void deleteFromMaxHeap(int arr[], int *n, int value) {
    int i;
    for (i = 0; i < *n; i++) {
        if (arr[i] == value)
            break;
    }

    if (i == *n) {
        printf("Element not found in Max Heap!\n");
        return;
    }

    arr[i] = arr[*n - 1];
    (*n)--;

    buildMaxHeap(arr, *n);
}

// Delete element from Min Heap
void deleteFromMinHeap(int arr[], int *n, int value) {
    for (i = 0; i < *n; i++) {
        if (arr[i] == value)
            break;
    }
}

```

```

    if (i == *n) {
        printf("Element not found in Min Heap!\n");
        return;
    }

    arr[i] = arr[*n - 1];
    (*n)--;

    buildMinHeap(arr, *n);
}

// Display Heap
void displayHeap(int arr[], int n) {
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

// Main
void main() {
    int arr[MAX], n, value, i;

    clrscr();
    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    // Max Heap
    buildMaxHeap(arr, n);
    printf("\nMax Heap: ");
    displayHeap(arr, n);

    printf("Enter element to delete from Max Heap: ");
    scanf("%d", &value);
    deleteFromMaxHeap(arr, &n, value);
    printf("Max Heap after deletion: ");
    displayHeap(arr, n);

    // Min Heap
    printf("\nEnter number of elements again: ");
    scanf("%d", &n);

```

```

printf("Enter elements: ");
for (i = 0; i < n; i++)
    scanf("%d", &arr[i]);

buildMinHeap(arr, n);
printf("\nMin Heap: ");
displayHeap(arr, n);

printf("Enter element to delete from Min Heap: ");
scanf("%d", &value);
deleteFromMinHeap(arr, &n, value);
printf("Min Heap after deletion: ");
displayHeap(arr, n);

getch();
}

```

Output :

Enter number of elements: 6
Enter elements: 20 15 30 5 10 25

Max Heap: 30 15 25 5 10 20
Enter element to delete from Max Heap: 25
Max Heap after deletion: 30 15 20 5 10

Enter number of elements again: 6
Enter elements: 20 15 30 5 10 25

Min Heap: 5 10 25 20 15 30
Enter element to delete from Min Heap: 10
Min Heap after deletion: 5 15 25 20 30

Experiments 4 :

Problem Statement:

"Implement BFT (Breadth First Traversal) and DFT (Depth First Traversal) for a given graph, when graph is represented by

a) Adjacency Matrix b) Adjacency List."

Aim

To implement Breadth First Traversal (BFT) and Depth First Traversal (DFT) of a graph using

1. Adjacency Matrix Representation
2. Adjacency List Representation

Algorithm

Breadth First Traversal (BFT)

1. Start from the source vertex.
2. Mark the source vertex as visited and put it in a queue.
3. While the queue is not empty:
 - Remove (dequeue) the front vertex from the queue.
 - Visit all unvisited adjacent vertices of this vertex:
 - Mark each as visited.
 - Add (enqueue) them into the queue for later processing.

Result: Vertices are visited level by level (like waves spreading out).

Depth First Traversal (DFT)

1. Start from the source vertex.
2. Mark it as visited and print it.
3. For each unvisited adjacent vertex:
 - Recursively apply DFT on that vertex

Sample code :

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 20

// ----- Queue for BFS -----
int queue[MAX], front = -1, rear = -1 ,i;
int visited[MAX];

// Enqueue
void enqueue(int v) {
```

```

    if (rear == MAX - 1) return;
    if (front == -1) front = 0;
    queue[++rear] = v;
}

// Dequeue
int dequeue() {
    if (front == -1 || front > rear) return -1;
    return queue[front++];
}

// BFS using adjacency matrix
void BFS_matrix(int adj[MAX][MAX], int n, int start) {
    for (i = 0; i < n; i++) visited[i] = 0;

    enqueue(start);
    visited[start] = 1;

    printf("BFS (Adjacency Matrix): ");
    while (front <= rear) {
        int v = dequeue();
        printf("%d ", v);

        for (i = 0; i < n; i++) {
            if (adj[v][i] == 1 && !visited[i]) {
                enqueue(i);
                visited[i] = 1;
            }
        }
    }
    printf("\n");
}

// DFS using adjacency matrix
void DFS_matrix(int adj[MAX][MAX], int n, int v) {
    printf("%d ", v);
    visited[v] = 1;

    for (i = 0; i < n; i++) {
        if (adj[v][i] == 1 && !visited[i])
            DFS_matrix(adj, n, i);
    }
}

```

```

// ----- Adjacency List Representation -----
struct Node {
    int vertex;
    struct Node* next;
};

struct Node* adjList[MAX];

// Add edge for adjacency list
void addEdge(int u, int v) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->vertex = v;
    newNode->next = adjList[u];
    adjList[u] = newNode;
}

// BFS using adjacency list
void BFS_list(int n, int start) {
    struct Node *temp;
    for (i = 0; i < n; i++) visited[i] = 0;
    front = rear = -1;

    enqueue(start);
    visited[start] = 1;

    printf("BFS (Adjacency List): ");
    while (front <= rear) {
        int v = dequeue();
        printf("%d ", v);

        temp = adjList[v];
        while (temp) {
            if (!visited[temp->vertex]) {
                enqueue(temp->vertex);
                visited[temp->vertex] = 1;
            }
            temp = temp->next;
        }
    }
    printf("\n");
}

// DFS using adjacency list
void DFS_list(int v) {

```

```

    struct Node *temp;
    printf("%d ", v);
    visited[v] = 1;

    temp = adjList[v];
    while (temp) {
        if (!visited[temp->vertex])
            DFS_list(temp->vertex);
        temp = temp->next;
    }
}

// ----- Main -----
void main() {
    int n, edges, u, v, i, j;
    int adj[MAX][MAX];
    int start;

    clrscr();
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    // Initialize adjacency matrix
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            adj[i][j] = 0;

    // Initialize adjacency list
    for (i = 0; i < n; i++)
        adjList[i] = NULL;

    printf("Enter number of edges: ");
    scanf("%d", &edges);

    printf("Enter edges (u v):\n");
    for (i = 0; i < edges; i++) {
        scanf("%d %d", &u, &v);
        adj[u][v] = adj[v][u] = 1; // Undirected graph
        addEdge(u, v);
        addEdge(v, u);
    }

    printf("Enter starting vertex: ");
    scanf("%d", &start);

```

```

// BFS and DFS using adjacency matrix
front = rear = -1;
BFS_matrix(adj, n, start);

for (i = 0; i < n; i++) visited[i] = 0;
printf("DFS (Adjacency Matrix): ");
DFS_matrix(adj, n, start);
printf("\n");

// BFS and DFS using adjacency list
BFS_list(n, start);

for (i = 0; i < n; i++) visited[i] = 0;
printf("DFS (Adjacency List): ");
DFS_list(start);
printf("\n");

getch();
}

```

Sample input & output :

Enter number of vertices: 5

Enter number of edges: 6

Enter edges (u v):

0 1

0 2

1 3

1 4

2 3

3 4

Enter starting vertex: 0

BFS (Adjacency Matrix): 0 1 2 3 4

DFS (Adjacency Matrix): 0 1 3 2 4

BFS (Adjacency List): 0 2 1 4 3

DFS (Adjacency List): 0 2 3 4 1

Experiments 5 :

Problem statement :

Write a program for finding the bi-connected components in a given graph.

Aim :

To find and display all biconnected components in a given graph using Depth First Search (DFS).

Algorithm (Tarjan's Algorithm for Biconnected Components):

1. Initialize:

- time = 0
- Arrays: disc[] (discovery times), low[] (earliest visited vertex reachable), parent[]
- Stack to store edges of DFS tree.

2. DFS Traversal:

- For every unvisited vertex u:
 - disc[u] = low[u] = ++time
 - For each adjacent vertex v of u:
 - If v is not visited:
 - Push edge (u,v) to stack
 - parent[v] = u
 - Recurse DFS(v)
 - low[u] = min(low[u], low[v])
 - If low[v] ≥ disc[u]:
 - Pop edges from stack until (u,v) → this is a biconnected component
 - Else if v != parent[u]:
 - low[u] = min(low[u], disc[v])
 - Push (u,v) to stack

3. After DFS:

- All edges in stack form remaining BCCs.

Sample code :

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 20 // Maximum vertices

typedef struct {
    int u, v;
} Edge;

int V, E;
```

```

int timeDFS = 0;
int disc[MAX], low[MAX], parent[MAX];
int stackTop = -1;
Edge stack[MAX * MAX];
int visited[MAX];

// Function to push edge on stack
void pushEdge(int u, int v) {
    stack[++stackTop].u = u;
    stack[stackTop].v = v;
}

// Function to print Biconnected Component
void printBCC(int u, int v) {
    printf("Biconnected Component: ");
    while (1) {
        Edge e = stack[stackTop--];
        printf("(%d-%d) ", e.u, e.v);
        if (e.u == u && e.v == v) break;
    }
    printf("\n");
}

// DFS to find Biconnected Components
void DFS(int u, int graph[MAX][MAX]) {
    int children = 0;
    int v;
    disc[u] = low[u] = ++timeDFS;

    for (v = 0; v < V; v++) {
        if (graph[u][v]) {
            if (disc[v] == -1) { // If v is not visited
                children++;
                parent[v] = u;
                pushEdge(u, v);
                DFS(v, graph);

                low[u] = (low[u] < low[v]) ? low[u] : low[v];

                // If u is articulation point, pop edges
                if ((parent[u] == -1 && children > 1) ||
                    (parent[u] != -1 && low[v] >= disc[u])) {
                    printBCC(u, v);
                }
            } else if (v != parent[u] && disc[v] < disc[u]) {
                low[u] = (low[u] < disc[v]) ? low[u] : disc[v];
                pushEdge(u, v);
            }
        }
    }
}

void main() {
    int graph[MAX][MAX] = {0};
    int i, u, v;
    clrscr();
    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter number of edges: ");
    scanf("%d", &E);
}

```

```

printf("Enter edges (u v) with vertices numbered from 0 to %d:\n", V - 1);
for (i = 0; i < E; i++) {
    scanf("%d %d", &u, &v);
    graph[u][v] = graph[v][u] = 1;
}

// Initialize arrays
for (i = 0; i < V; i++) {
    disc[i] = -1;
    low[i] = -1;
    parent[i] = -1;
}

printf("\nBiconnected Components in the graph:\n");
for (i = 0; i < V; i++) {
    if (disc[i] == -1)
        DFS(i, graph);

    // Print remaining edges if any
    if (stackTop != -1) {
        printBCC(stack[0].u, stack[0].v);
    }
}
getch();
}

```

SAMPLE OUTPUT :

Enter number of vertices: 5

Enter number of edges: 5

Enter edges (u v) with vertices numbered from 0 to 4:

1 0

0 2

1 2

3 4

0 3

Biconnected Components in the graph:

Biconnected Component: (3-4)

Biconnected Component: (0-3)

Biconnected Component: (2-0) (1-2) (0-1)

Experiments 6:

Problem statement :

Implement **Quick Sort** and **Merge Sort** algorithms and compare their execution times for **Best Case, Average Case, and Worst Case** inputs with different input sizes.

Aim

To study and compare the performance of **Quick Sort** and **Merge Sort** by executing them on arrays of various sizes under **Best, Average, and Worst case scenarios**, and observe their execution times.

Algorithm

Quick Sort Algorithm

1. Choose a pivot element (here: last element of the array).
2. Partition the array such that elements smaller than the pivot are on the left, and greater elements are on the right.
3. Recursively apply the above steps to left and right subarrays.
4. Base case: If the subarray has 1 or 0 elements, return.
 - **Best Case:** Balanced partition ($O(n \log n)$).
 - **Worst Case:** Already sorted/reverse sorted array ($O(n^2)$).

Merge Sort Algorithm

1. If array has 1 element, return (base case).
2. Divide the array into two halves.
3. Recursively sort the left half and right half.
4. Merge the two sorted halves into a single sorted array.
 - **Best, Average, Worst Case:** Always $O(n \log n)$.

Sample code 🙄 [WORK WITH ONLINE COMPILER](#)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// Function to swap two integers
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

```
// Partition function for QuickSort
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // choosing last element as pivot
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}
```

```
// QuickSort function
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

```
EX5.CPP
// Merge function for MergeSort
void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1;
    int n2 = r - m;

    int *L = (int *)malloc(n1 * sizeof(int));
    int *R = (int *)malloc(n2 * sizeof(int));

    for (int i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }
}
```

```

        while (i < n1)
            arr[k++] = L[i++];

        while (j < n2)
            arr[k++] = R[j++];

        free(L);
        free(R);
    }

    // MergeSort function
    void mergeSort(int arr[], int l, int r) {
        if (l < r) {
            int m = (l + r) / 2;
            mergeSort(arr, l, m);
            mergeSort(arr, m + 1, r);
            merge(arr, l, m, r);
        }
    }
}

```

```

// Utility functions to generate test cases
void generateBestCase(int arr[], int n) {
    for (int i = 0; i < n; i++)
        arr[i] = i; // sorted array
}

void generateWorstCase(int arr[], int n) {
    for (int i = 0; i < n; i++)
        arr[i] = n - i; // reverse sorted array
}

void generateAverageCase(int arr[], int n) {
    for (int i = 0; i < n; i++)
        arr[i] = rand() % n; // random array
}

// Function to copy array
void copyArray(int src[], int dest[], int n) {
    for (int i = 0; i < n; i++)
        dest[i] = src[i];
}

```

```

// Function to measure execution time
double measureQuickSort(int arr[], int n) {
    clock_t start, end;
    start = clock();
    quickSort(arr, 0, n - 1);
    end = clock();
    return ((double)(end - start)) / CLOCKS_PER_SEC;
}

double measureMergeSort(int arr[], int n) {
    clock_t start, end;
    start = clock();
    mergeSort(arr, 0, n - 1);
    end = clock();
    return ((double)(end - start)) / CLOCKS_PER_SEC;
}

```

```

int main() {
    srand(time(0));
    int sizes[] = {1000, 5000, 10000, 20000}; // different input sizes
    int nSizes = sizeof(sizes) / sizeof(sizes[0]);

    for (int s = 0; s < nSizes; s++) {
        int n = sizes[s];
        int *arr = (int *)malloc(n * sizeof(int));
        int *copy = (int *)malloc(n * sizeof(int));

        printf("\n--- Input size: %d ---\n", n);

        // Best Case
        generateBestCase(arr, n);
        copyArray(arr, copy, n);
        printf("Best Case:\n");
        printf(" QuickSort: %f sec\n", measureQuickSort(copy, n));
        copyArray(arr, copy, n);
        printf(" MergeSort: %f sec\n", measureMergeSort(copy, n));

        // Average Case
        generateAverageCase(arr, n);
        copyArray(arr, copy, n);
        printf("Average Case:\n");
        printf(" QuickSort: %f sec\n", measureQuickSort(copy, n));
        copyArray(arr, copy, n);
        printf(" MergeSort: %f sec\n", measureMergeSort(copy, n));

        // Worst Case
        generateWorstCase(arr, n);
        copyArray(arr, copy, n);
        printf("Worst Case:\n");
        printf(" QuickSort: %f sec\n", measureQuickSort(copy, n));
        copyArray(arr, copy, n);
        printf(" MergeSort: %f sec\n", measureMergeSort(copy, n));

        free(arr);
        free(copy);
    }
    return 0;
}

```

Example output :

--- Input size: 1000 ---

Best Case:

QuickSort: 0.000095 sec

MergeSort: 0.000082 sec

Average Case:

QuickSort: 0.000121 sec

MergeSort: 0.000085 sec

Worst Case:

QuickSort: 0.000500 sec

MergeSort: 0.000087 sec

--- Input size: 5000 ---

Best Case:

QuickSort: 0.000610 sec

MergeSort: 0.000500 sec

Average Case:

QuickSort: 0.000750 sec

MergeSort: 0.000520 sec

Worst Case:

QuickSort: 0.005200 sec

MergeSort: 0.000550 sec

--- Input size: 10000 ---

Best Case:

QuickSort: 0.001400 sec

MergeSort: 0.001200 sec

Average Case:

QuickSort: 0.001650 sec

MergeSort: 0.001210 sec

Worst Case:

QuickSort: 0.020300 sec

MergeSort: 0.001250 sec

--- Input size: 20000 ---

Best Case:

QuickSort: 0.003100 sec

MergeSort: 0.002500 sec

Average Case:

QuickSort: 0.003800 sec

MergeSort: 0.002600 sec

Worst Case:

QuickSort: 0.080000 sec

MergeSort: 0.002700 sec

Result: thus, the above program has been executed successfully .

Experiments 7:

Problem statement :

To compare the performance of **Single Source Shortest Path (SSSP)** using **Greedy method (Dijkstra's Algorithm)** when the graph is represented by **Adjacency Matrix** and **Adjacency List**.

Aim

To implement and analyze the efficiency of Dijkstra's Algorithm for finding shortest paths from a single source vertex to all other vertices using adjacency matrix and adjacency list representations.

Algorithm (Dijkstra's Algorithm – Greedy Method)

1. Input the graph with n vertices and source vertex s .
2. Initialize distance array $dist[]$ with ∞ (infinity), and set $dist[s] = 0$.
3. Mark all vertices as unvisited ($sptSet[] = 0$).
4. Repeat for $n - 1$ iterations:
 - Select the unvisited vertex u with the minimum distance value.
 - Mark u as visited.
 - Update the distance of all adjacent vertices v of u :
 - if $dist[u] + weight(u, v) < dist[v]$ then
 - $dist[v] = dist[u] + weight(u, v)$
5. Print the shortest distances from the source to all vertices.
6. Perform the above steps once using **adjacency matrix** and once using **adjacency list**.

SAMPLE CODE :

```
#include <stdio.h>
#include <conio.h>
#include <limits.h>
#include <stdlib.h>

#define MAX 10 /* Maximum number of vertices */

int n; /* number of vertices */

/* ----- Find minimum distance vertex ----- */
int minDistance(int dist[], int sptSet[]) {
    int min = INT_MAX, min_index = -1, v;
    for (v = 0; v < n; v++) {
        if (sptSet[v] == 0 && dist[v] <= min) {
            min = dist[v];
            min_index = v;
        }
    }
    return min_index;
}
```

```

}

/* ----- Dijkstra using Adjacency Matrix ----- */
void dijkstra_matrix(int graph[MAX][MAX], int src) {
    int dist[MAX]; /* shortest distance array */
    int sptSet[MAX]; /* 1 if vertex included */
    int count, v, u;

    for (v = 0; v < n; v++) {
        dist[v] = INT_MAX;
        sptSet[v] = 0;
    }

    dist[src] = 0;

    for (count = 0; count < n - 1; count++) {
        u = minDistance(dist, sptSet);
        sptSet[u] = 1;

        for (v = 0; v < n; v++) {
            if (!sptSet[v] && graph[u][v] && dist[u] != INT_MAX
                && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    printf("Vertex \t Distance from Source\n");
    for (v = 0; v < n; v++) {
        printf("%d \t %d\n", v, dist[v]);
    }
}

/* ----- Dijkstra using Adjacency List (from matrix) ----- */
void dijkstra_list(int graph[MAX][MAX], int src) {
    struct Node {
        int vertex;
        int weight;
        struct Node *next;
    };

    struct Node *adjList[MAX];
    int i, j;

    /* Initialize adjacency list */

```

```

for (i = 0; i < n; i++) {
    adjList[i] = NULL;
}

/* Convert matrix to list */
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        if (graph[i][j] != 0) {
            struct Node *newNode;
            newNode = (struct Node *) malloc(sizeof(struct Node));
            newNode->vertex = j;
            newNode->weight = graph[i][j];
            newNode->next = adjList[i];
            adjList[i] = newNode;
        }
    }
}

/* Dijkstra on adjacency list */
int dist[MAX], sptSet[MAX], count, u, v;
for (i = 0; i < n; i++) {
    dist[i] = INT_MAX;
    sptSet[i] = 0;
}
dist[src] = 0;

for (count = 0; count < n - 1; count++) {
    u = minDistance(dist, sptSet);
    sptSet[u] = 1;

    struct Node *temp = adjList[u];
    while (temp != NULL) {
        v = temp->vertex;
        if (!sptSet[v] && dist[u] != INT_MAX
            && dist[u] + temp->weight < dist[v]) {
            dist[v] = dist[u] + temp->weight;
        }
        temp = temp->next;
    }
}

printf("Vertex \t Distance from Source\n");
for (i = 0; i < n; i++) {
    printf("%d \t %d\n", i, dist[i]);
}

```

```

}

/* ----- Main Function ----- */
void main() {
    int graph[MAX][MAX];
    int i, j, src;

    clrscr();

    printf("Enter number of vertices (max %d): ", MAX);
    scanf("%d", &n);

    printf("Enter adjacency matrix (use 0 for no edge):\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    printf("Enter source vertex (0 to %d): ", n - 1);
    scanf("%d", &src);

    printf("\n--- Using Adjacency Matrix ---\n");
    dijkstra_matrix(graph, src);

    printf("\n--- Using Adjacency List ---\n");
    dijkstra_list(graph, src);

    getch();
}

```

Sample Input

```

Enter number of vertices (max 10): 5
Enter adjacency matrix (use 0 for no edge):
0 10 0 5 0
0 0 1 2 0
0 0 0 0 4
0 3 9 0 2
7 0 6 0 0

```

```

Enter source vertex (0 to 4): 0

```

Output (Adjacency Matrix Representation)

--- Using Adjacency Matrix ---

Vertex Distance from Source

0	0
1	8
2	9
3	5
4	7

Output (Adjacency List Representation)

--- Using Adjacency List ---

Vertex Distance from Source

0	0
1	8
2	9
3	5
4	7

Result

The program for **Single Source Shortest Path using Greedy method (Dijkstra's Algorithm)** was successfully executed using both **Adjacency Matrix** and **Adjacency List** representations.

The shortest distances from the given source vertex to all other vertices were obtained correctly.

Thus, the experiment was executed successfully and the performance comparison was observed

Experiment 8 :

Problem Statement

Implement the **Job Sequencing with Deadlines** problem using the **Greedy strategy** to find the optimal sequence of jobs that maximizes total profit while meeting their respective deadlines.

Aim

To design and implement a C program for **Job Sequencing with Deadlines** using the **Greedy method**, and determine the job order that gives **maximum profit** within the given deadlines.

Algorithm

1. **Start**
 2. Read the number of jobs n.
 3. For each job, input **Job ID**, **Deadline**, and **Profit**.
 4. Sort all jobs in **descending order of profit**.
 5. Find the **maximum deadline** among all jobs.
 6. Create an array of time slots (all initially free).
 7. For each job in the sorted list:
 - Check for a **free slot** from its deadline backward.
 - If a slot is found, assign the job to that slot.
 8. Print the **job sequence** and **total profit** obtained.
 9. **Stop**
-

Program (Turbo C Compatible)

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
struct Job {  
    char id;    // Job ID  
    int deadline; // Deadline of job  
    int profit;  // Profit if job is completed before or on deadline  
};
```

```
void jobSequencing(struct Job jobs[], int n) {
```

```
int i, j, maxDeadline = 0, totalProfit = 0;
```

```
// Find maximum deadline
```

```
for (i = 0; i < n; i++) {  
    if (jobs[i].deadline > maxDeadline)  
        maxDeadline = jobs[i].deadline;  
}
```

```
// Sort jobs in descending order of profit
```

```
for (i = 0; i < n - 1; i++) {  
    for (j = i + 1; j < n; j++) {  
        if (jobs[i].profit < jobs[j].profit) {  
            struct Job temp = jobs[i];  
            jobs[i] = jobs[j];  
            jobs[j] = temp;  
        }  
    }  
}
```

```
int slot[maxDeadline];
```

```
char result[maxDeadline];
```

```
for (i = 0; i < maxDeadline; i++)  
    slot[i] = 0; // All slots are initially free
```

```
// Assign jobs to slots
```

```
for (i = 0; i < n; i++) {  
    for (j = jobs[i].deadline - 1; j >= 0; j--) {  
        if (slot[j] == 0) {  
            slot[j] = 1;  
            result[j] = jobs[i].id;  
            totalProfit += jobs[i].profit;  
            break;  
        }  
    }  
}
```

```

    }

    // Display the results
    printf("\nJob Sequence: ");
    for (i = 0; i < maxDeadline; i++) {
        if (slot[i])
            printf("%c ", result[i]);
    }

    printf("\nTotal Profit: %d", totalProfit);
}

void main() {
    int n, i;
    struct Job jobs[10];

    clrscr();
    printf("Enter number of jobs: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("\nEnter Job ID, Deadline, and Profit for Job %d: ", i + 1);
        fflush(stdin);
        scanf(" %c %d %d", &jobs[i].id, &jobs[i].deadline, &jobs[i].profit);
    }

    printf("\n--- Job Sequencing with Deadlines (Greedy Strategy) ---\n");
    jobSequencing(jobs, n);

    printf("\n\nProgram executed successfully!");
    getch();
}

```

Sample Input and Output

Input:

Enter number of jobs: 5

Enter Job ID, Deadline, and Profit for Job 1: A 2 100

Enter Job ID, Deadline, and Profit for Job 2: B 1 19

Enter Job ID, Deadline, and Profit for Job 3: C 2 27

Enter Job ID, Deadline, and Profit for Job 4: D 1 25

Enter Job ID, Deadline, and Profit for Job 5: E 3 15

Output:

--- Job Sequencing with Deadlines (Greedy Strategy) ---

Job Sequence: C A E

Total Profit: 142

Program executed successfully!

Result

The program successfully implements the **Job Sequencing with Deadlines** using the **Greedy method**. The obtained **optimal job sequence** yields a **maximum total profit** of **142** for the given input.

Experiment 9: 0/1 Knapsack Problem using Dynamic Programming

Problem Statement

Write a program to solve the **0/1 Knapsack problem** using the **Dynamic Programming approach** to determine the maximum total profit (value) that can be obtained by selecting items within a given capacity constraint.

Aim

To design and implement a **C program** to solve the **0/1 Knapsack problem** using **Dynamic Programming**, thereby finding the most profitable combination of items that fits within the knapsack's weight limit.

Algorithm

1. **Start**
 2. Read the number of items n and knapsack capacity W .
 3. Input the **weights** and **profits/values** of all items.
 4. Construct a table $K[n+1][W+1]$ where $K[i][w]$ represents the maximum profit using the first i items and capacity w .
 5. Initialize the first row and column to 0 (base cases).
 6. For each item i and each capacity w :
 - If $\text{weight}[i] \leq w$ then
$$K[i][w] = \max(\text{value}[i] + K[i-1][w - \text{weight}[i]], K[i-1][w])$$
 - Else
$$K[i][w] = K[i-1][w]$$
 7. After filling the table, $K[n][W]$ gives the **maximum profit**.
 8. Backtrack to find which items are included in the optimal solution.
 9. **Stop**
-

Program (Turbo C Compatible)

```
#include <stdio.h>
#include <conio.h>
```

```
int max(int a, int b) {
    return (a > b) ? a : b;
```

```

}

void knapsack(int n, int weight[], int profit[], int capacity) {
    int i, w;
    int K[20][20]; // DP table

    // Build table K[][] in bottom-up manner
    for (i = 0; i <= n; i++) {
        for (w = 0; w <= capacity; w++) {
            if (i == 0 || w == 0)
                K[i][w] = 0;
            else if (weight[i-1] <= w)
                K[i][w] = max(profit[i-1] + K[i-1][w - weight[i-1]], K[i-1][w]);
            else
                K[i][w] = K[i-1][w];
        }
    }

    printf("\nMaximum Profit = %d", K[n][capacity]);

    // Display selected items
    printf("\nSelected Items: ");
    w = capacity;
    for (i = n; i > 0 && w > 0; i--) {
        if (K[i][w] != K[i-1][w]) {
            printf("Item%d ", i);
            w = w - weight[i-1];
        }
    }
}

void main() {
    int n, i, capacity;
    int weight[10], profit[10];

    clrscr();
    printf("Enter number of items: ");
    scanf("%d", &n);

    printf("Enter weights of items: ");
    for (i = 0; i < n; i++)
        scanf("%d", &weight[i]);

    printf("Enter profits of items: ");
    for (i = 0; i < n; i++)

```

```
scanf("%d", &profit[i]);

printf("Enter capacity of knapsack: ");
scanf("%d", &capacity);

printf("\n--- 0/1 Knapsack using Dynamic Programming ---\n");
knapsack(n, weight, profit, capacity);

printf("\n\nProgram executed successfully!");
getch();
}
```

Sample Input and Output

Input:

Enter number of items: 4

Enter weights of items: 2 3 4 5

Enter profits of items: 3 4 5 6

Enter capacity of knapsack: 5

Output:

--- 0/1 Knapsack using Dynamic Programming ---

Maximum Profit = 7

Selected Items: Item2 Item1

Program executed successfully!

Result

The program successfully implements the **0/1 Knapsack Problem** using **Dynamic Programming**. For the given input, the **maximum profit** obtained is **7**, achieved by selecting **Item 2 and Item 1**.

Experiment 10: N-Queens Problem using Backtracking

Problem Statement

Write a program to solve the **N-Queens Problem** using the **Backtracking technique**, such that no two queens attack each other (i.e., no two queens share the same row, column, or diagonal).

Aim

To design and implement a **C program** for the **N-Queens Problem** using the **Backtracking approach** to place N queens on an $N \times N$ chessboard so that no two queens threaten each other.

Algorithm

1. **Start**
 2. Place a queen in the first column of the first row.
 3. For each column in the current row:
 - Check if placing the queen at (row, col) is safe (no queen in same column, and no queen in diagonals).
 - If safe, place the queen and move to the next row.
 - If placing leads to a solution, return true.
 - Else, remove the queen (backtrack) and try next column.
 4. If all queens are placed successfully, print the board configuration.
 5. If no valid position exists, backtrack to previous row.
 6. **Stop**
-

Program (Turbo C Compatible)

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
#define MAX 10
```

```
int board[MAX], n;
```

```
int place(int row, int column) {
```

```

int i;
for (i = 1; i <= row - 1; i++) {
    if (board[i] == column ||
        abs(board[i] - column) == abs(i - row))
        return 0;
}
return 1;
}

```

```

void printBoard() {
    int i, j;
    printf("\nSolution:\n");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            if (board[i] == j)
                printf(" Q ");
            else
                printf(" . ");
        }
        printf("\n");
    }
}

```

```

void nQueen(int row) {
    int column;
    for (column = 1; column <= n; column++) {
        if (place(row, column)) {
            board[row] = column;
            if (row == n)
                printBoard();
            else
                nQueen(row + 1);
        }
    }
}

```

```

}

void main() {
    clrscr();
    printf("Enter number of Queens: ");
    scanf("%d", &n);
    nQueen(1);
    printf("\nProgram executed successfully!");
    getch();
}

```

Sample Input and Output

Input:

Enter number of Queens: 4

Output:

Solution:

```

. Q . .
. . . Q
Q . . .
. . Q .

```

Solution:

```

. . Q .
Q . . .
. . . Q
. Q . .

```

Program executed successfully!

Result

The program successfully implements the **N-Queens Problem** using **Backtracking**.

For N = 4, the program finds and displays **two valid solutions** in which no two queens attack each other.

Experiment No: 11

Aim:

To implement the **0/1 Knapsack problem** using **Backtracking strategy** to obtain the maximum profit without exceeding the knapsack capacity.

Problem Statement:

Given n items with individual **weights** and **profits**, determine the most profitable subset of items that can fit in a knapsack of given capacity W, using **backtracking**.

Algorithm:

1. Start with the first item.
 2. For each item, there are two choices — include or exclude it.
 3. Include the item only if it does not exceed the knapsack capacity.
 4. Compute total profit and weight after each decision.
 5. Backtrack if total weight exceeds capacity.
 6. Keep track of the **maximum profit** among all valid combinations.
 7. End after exploring all combinations.
-

C Program (Turbo C2 Compatible):

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
int n;
```

```
float w[10], p[10], W;
```

```
float maxProfit = 0, cw = 0, cp = 0;
```

```
void knapsack(int i)
```

```
{
```

```
    if (i > n)
```

```
    {
```

```
        if (cp > maxProfit)
```

```
            maxProfit = cp;
```

```
        return;
```

```
    }
```

```
    // Include item i
```

```
    if (cw + w[i] <= W)
```

```

{
    cw = cw + w[i];
    cp = cp + p[i];
    knapsack(i + 1);
    cw = cw - w[i];
    cp = cp - p[i];
}

// Exclude item i
knapsack(i + 1);
}

void main()
{
    int i;
    clrscr();

    printf("Enter number of items: ");
    scanf("%d", &n);

    printf("Enter weights of %d items:\n", n);
    for (i = 1; i <= n; i++)
        scanf("%f", &w[i]);

    printf("Enter profits of %d items:\n", n);
    for (i = 1; i <= n; i++)
        scanf("%f", &p[i]);

    printf("Enter capacity of knapsack: ");
    scanf("%f", &W);

    knapsack(1);

    printf("\nMaximum profit = %.2f", maxProfit);

    getch();
}

```

Sample Input / Output:

Enter number of items: 3

Enter weights of 3 items:

2 3 4

Enter profits of 3 items:

1 2 5

Enter capacity of knapsack: 5

Maximum profit = 6.00

Result:

Thus, the **0/1 Knapsack Problem** was successfully implemented using the **Backtracking strategy** in **Turbo C2**, and the **maximum profit** was obtained correctly.

Experiment No: 12

Title:

Implement **Travelling Salesperson Problem (TSP)** using **Branch and Bound** approach.

Aim:

To solve the **Travelling Salesperson Problem** using the **Branch and Bound** technique and find the shortest possible route that visits each city exactly once and returns to the origin city.

Problem Statement:

Given a set of cities and the distances between each pair of cities, determine the minimum cost route that visits every city exactly once and returns to the starting point, using the **Branch and Bound** method.

Algorithm:

1. **Start** from an initial city.
2. **Compute lower bound** for the current partial path (using cost matrix).
3. **Select the next city** with the least additional cost that maintains a minimal total bound.
4. **Branch** by exploring possible routes recursively.

5. **Bound:** Prune (ignore) paths whose cost exceeds the current best-known cost.
 6. **Repeat** until all cities are visited once.
 7. **Return** to the starting city and report the **minimum total cost** and path.
-

C Program (Turbo C2 Compatible):

```
#include <stdio.h>
#include <conio.h>

#define MAX 10
#define INF 999

int n;
int cost[MAX][MAX];
int visited[MAX];
int minCost = INF;
int finalPath[MAX];
int tempPath[MAX];

void copyPath(int tempPath[], int finalPath[])
{
    int i;
    for (i = 0; i < n; i++)
        finalPath[i] = tempPath[i];
    finalPath[n] = tempPath[0];
}

void TSP(int city, int count, int costTillNow)
{
    int i;

    if (count == n && cost[city][0] > 0)
    {
        int totalCost = costTillNow + cost[city][0];
        if (totalCost < minCost)
        {
            minCost = totalCost;
            tempPath[count] = city;
            copyPath(tempPath, finalPath);
        }
        return;
    }

    for (i = 0; i < n; i++)
```

```

{
    if (!visited[i] && cost[city][i] > 0)
    {
        visited[i] = 1;
        tempPath[count] = city;
        TSP(i, count + 1, costTillNow + cost[city][i]);
        visited[i] = 0;
    }
}

void main()
{
    int i, j;
    clrscr();

    printf("Enter number of cities: ");
    scanf("%d", &n);

    printf("Enter cost matrix:\n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0)
                cost[i][j] = INF;
        }
    }

    for (i = 0; i < n; i++)
        visited[i] = 0;

    visited[0] = 1;
    TSP(0, 1, 0);

    printf("\nMinimum Cost: %d", minCost);
    printf("\nOptimal Path: ");
    for (i = 0; i <= n; i++)
        printf("%d ", finalPath[i]);

    getch();
}

```

Sample Input / Output:

Enter number of cities: 4

Enter cost matrix:

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Minimum Cost: 80

Optimal Path: 0 1 3 2 0

Result:

Thus, the **Travelling Salesperson Problem (TSP)** was successfully implemented using the **Branch and Bound approach** in **Turbo C2**, and the minimum cost tour was found.